Treewidth in Program Analysis

Giovanna Kobus Conrado

September 5, 2026

Overview

You may have heard me, Andreas, and other people in PL talk about treewidth.

What is treewidth?

Number that measures how simple a graph is.
Algorithms run faster on low-treewidth graphs.
E.g. 3-coloring is NP-complete, but can be solved in linear time for graphs of bounded treewidth.

Goals for this talk

You should be able to

- ◇ Identify low-treewidth graphs in your own research
- ◇ Identify problems that may be solved more efficiently in low-treewidth graphs
- ◇ Get some of your own algorithms to run faster!

Overview

What is treewidth

- Why trees are nice
- Constructing a graph of treewidth k
- Properties of low-treewidth graphs

Treewidth in program analysis

- Register Allocation
- Algebraic Program Analysis
- Robustness of Concurrent Programs

Trees

We love trees. Why?

Trees

We love trees. Why?

Pros

1. **NP-complete problems can be polynomial on trees** (coloring, smallest vertex cover, largest independent set)
2. **Polynomial problems can be more efficient on trees** (distance queries, dynamic connectivity)
3. **Tree automata allow us to define infinite sets of trees and verify properties of these sets** (reachability in program states)

Trees

We love trees. Why?

Pros

1. **NP-complete problems can be polynomial on trees** (coloring, smallest vertex cover, largest independent set)
2. **Polynomial problems can be more efficient on trees** (distance queries, dynamic connectivity)
3. **Tree automata allow us to define infinite sets of trees and verify properties of these sets** (reachability in program states)

Cons

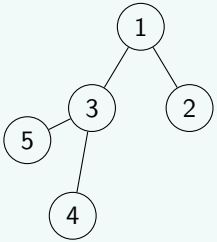
1. **They cannot represent anything complicated**

Treewidth is an attempt generalize trees to larger classes of graphs while keeping the pros.

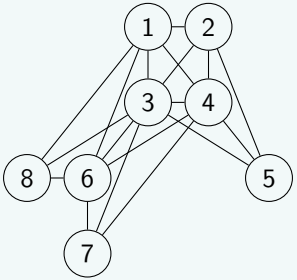
Treewidth

Treewidth is a number associated to a graph measuring how tree-like it is.

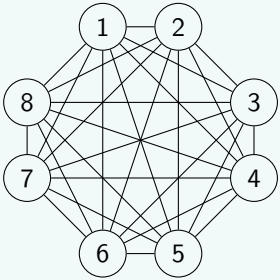
Treewidth 1



Treewidth 3



Treewidth $(n-1)$



Treewidth

First definition of treewidth: show how to build a tree then generalize it.

How to build a tree?

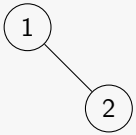
- Start with a vertex.
- Repeatedly add vertices v adjacent to some previously existing vertex u.

Tree

How to build a tree?

- Start with a vertex.
- Repeatedly add vertices v adjacent to some previously existing vertex u.

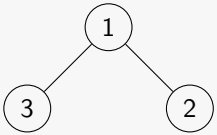
Tree



How to build a tree?

- Start with a vertex.
- Repeatedly add vertices v adjacent to some previously existing vertex u.

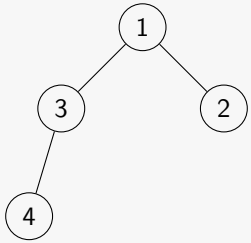
Tree



How to build a tree?

- Start with a vertex.
- Repeatedly add vertices v adjacent to some previously existing vertex u.

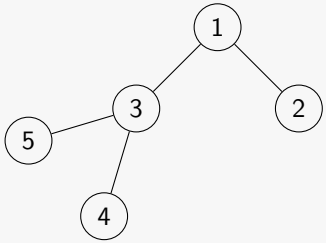
Tree



How to build a tree?

- Start with a vertex.
- Repeatedly add vertices v adjacent to some previously existing vertex u.

Tree



How do we generalize this process?

Make the tree "fatter": our new nodes ("bags") will now represent a small clique. We add new nodes while keeping the tree-of-cliques structure.

k-tree

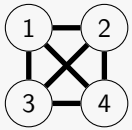
A k-tree is a graph that can be created by the following procedure:

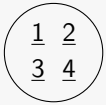
- Start with a $(k+1)$-clique.
- Repeatedly add vertices v adjacent to U such that $|U| = k$ and U is a k-clique (therefore forming another $(k+1)$-clique).

How do we generalize trees?

- ◇ Start with a $(k+1)$-clique.
- ◇ Repeatedly add vertices v adjacent to U such that $|U| = k$ and U is a k-clique (therefore forming another $(k+1)$-clique).

3-tree

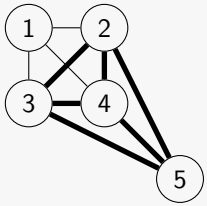


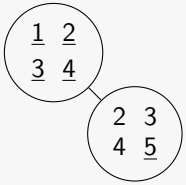


How do we generalize trees?

- Start with a $(k+1)$-clique.
- Repeatedly add vertices v adjacent to U such that $|U| = k$ and U is a k-clique (therefore forming another $(k+1)$-clique).

3-tree

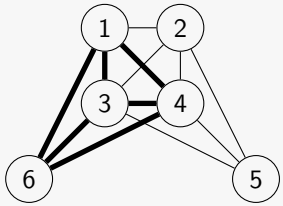


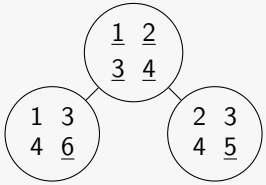


How do we generalize trees?

- Start with a $(k+1)$-clique.
- Repeatedly add vertices v adjacent to U such that $|U| = k$ and U is a k-clique (therefore forming another $(k+1)$-clique).

3-tree

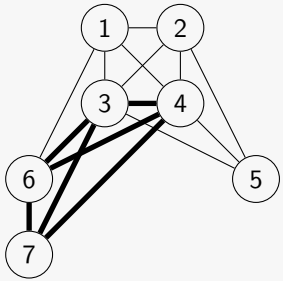


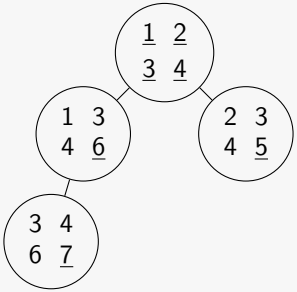


How do we generalize trees?

- Start with a $(k+1)$-clique.
- Repeatedly add vertices v adjacent to U such that $|U| = k$ and U is a k-clique (therefore forming another $(k+1)$-clique).

3-tree

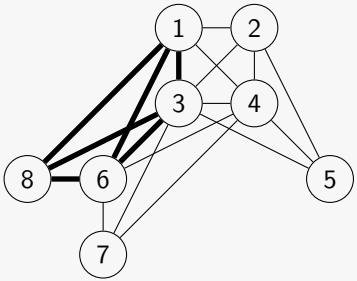


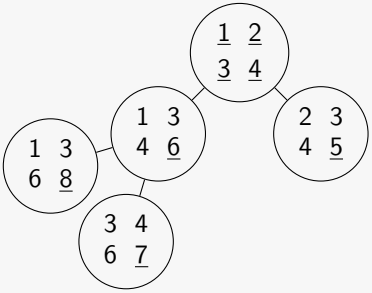


How do we generalize trees?

- Start with a $(k+1)$-clique.
- Repeatedly add vertices v adjacent to U such that $|U| = k$ and U is a k-clique (therefore forming another $(k+1)$-clique).

3-tree

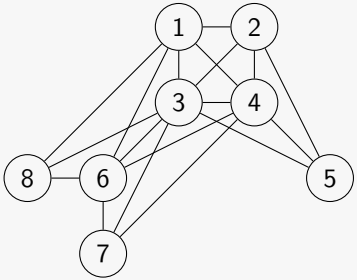


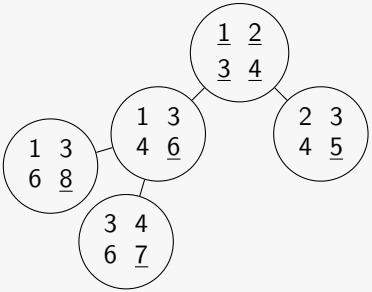


How do we generalize trees?

- The graph G formed is a k-tree
- The respective tree of bags is a **tree decomposition of G of width k**

3-tree and tree decomposition of width 3

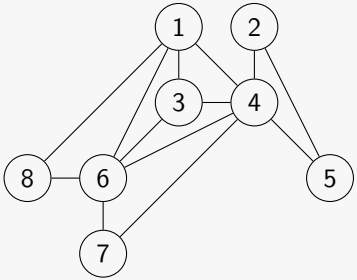


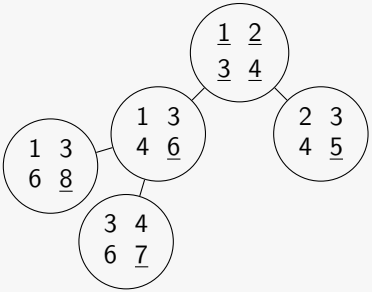


How do we generalize trees?

- A partial k-tree is a subgraph of a k-tree
- Graphs of treewidth at most k are the partial k-trees

Partial 3-tree and tree decomposition of width 3





Treewidth

This is not very helpful for algorithm design, let us look at an equivalent definition.

Characterizing partial k-trees

Tree decomposition

A tree decomposition of a graph $G = (V, E)$ is a tree T of bags X_i such that

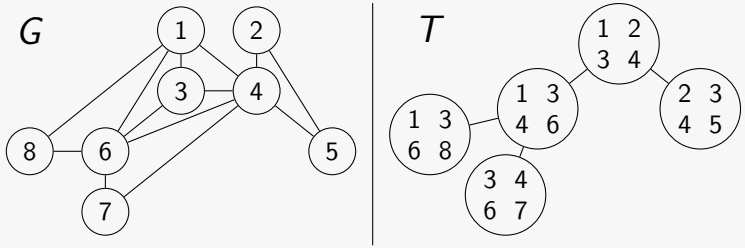
1. Every vertex is in some bag $(\forall v \in V, \exists X_i | v \in X_i)$
2. Every edge is in some bag $(\forall (u, v) \in E, \exists X_i | u, v \in X_i)$
3. Every vertex appears in a connected subgraph of T

Characterizing partial k-trees

Tree decomposition

A tree decomposition of a graph $G = (V, E)$ is a tree T of bags X_i such that

1. Every vertex is in some bag $(\forall v \in V, \exists X_i | v \in X_i)$
2. Every edge is in some bag $(\forall (u, v) \in E, \exists X_i | u, v \in X_i)$
3. Every vertex appears in a connected subgraph of T

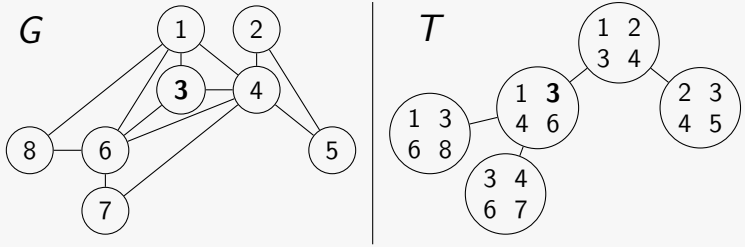


Characterizing partial k-trees

Tree decomposition

A tree decomposition of a graph $G = (V, E)$ is a tree T of bags X_i such that

1. **Every vertex is in some bag** $(\forall v \in V, \exists X_i | v \in X_i)$
2. Every edge is in some bag $(\forall (u, v) \in E, \exists X_i | u, v \in X_i)$
3. Every vertex appears in a connected subgraph of T

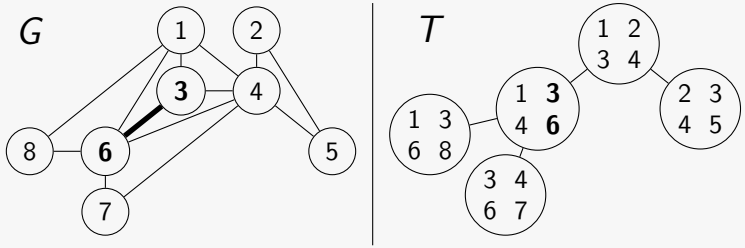


Characterizing partial k-trees

Tree decomposition

A tree decomposition of a graph $G = (V, E)$ is a tree T of bags X_i such that

1. Every vertex is in some bag $(\forall v \in V, \exists X_i | v \in X_i)$
2. **Every edge is in some bag** $(\forall (u, v) \in E, \exists X_i | u, v \in X_i)$
3. Every vertex appears in a connected subgraph of T

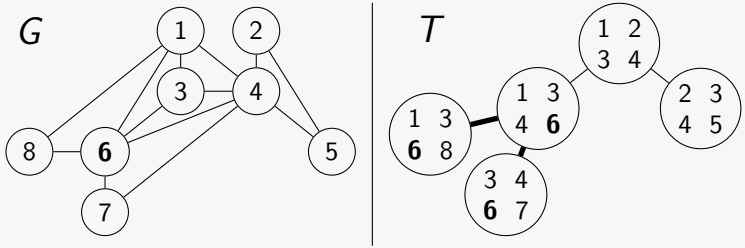


Characterizing partial k-trees

Tree decomposition

A tree decomposition of a graph $G = (V, E)$ is a tree T of bags X_i such that

1. Every vertex is in some bag $(\forall v \in V, \exists X_i | v \in X_i)$
2. Every edge is in some bag $(\forall (u, v) \in E, \exists X_i | u, v \in X_i)$
3. **Every vertex appears in a connected subgraph of T**



Characterizing partial k-trees

Treewidth

A tree decomposition of a graph $G = (V, E)$ is a tree T of bags X_i such that

1. Every vertex is in some bag $(\forall v \in V, \exists X_i | v \in X_i)$
2. Every edge is in some bag $(\forall (u, v) \in E, \exists X_i | u, v \in X_i)$
3. Every vertex appears in a connected subgraph of T

The width of T is $\max_i |X_i| - 1$.

The treewidth of a graph is the lowest k such that G has a tree decomposition of width k.

What properties do bounded-treewidth graphs have?

Sparsity

How many edges can a graph G of treewidth k have?

Sparsity

How many edges can a graph G of treewidth k have?

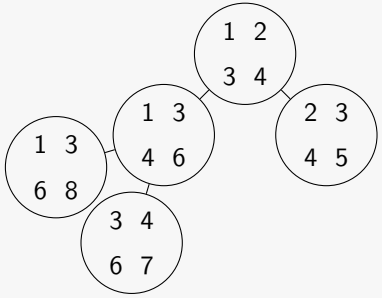
By our "construction" method, every time I add a vertex I also add k edges.

There are $O(k)$ edges per vertex.

In total, there are $O(n \cdot k)$ edges.

Graphs of bounded treewidth are sparse.

Tree decomposition of width 3



However, there are sparse graphs of unbounded treewidth.

Small separators

Separators are small sets of vertices that, when removed from the graph, split it into disconnected pieces.

Every bag X_i in T is a separator for G.

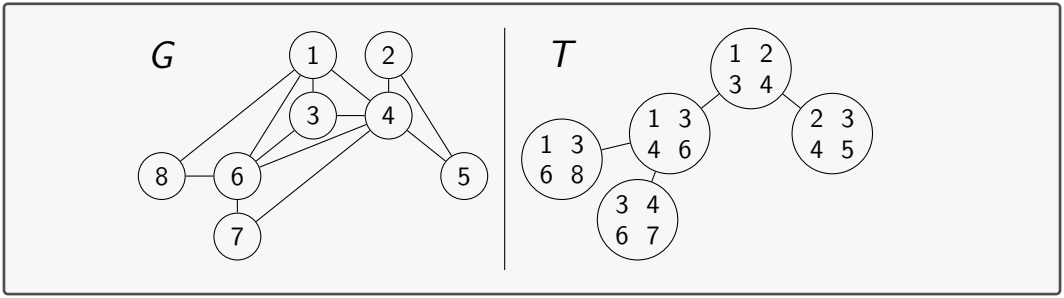
You can also always separate G into small ($< |V|/2$) parts.

Small separators

Separators are small sets of vertices that, when removed from the graph, split it into disconnected pieces.

Every bag X_i in T is a separator for G.

You can also always separate G into small ($< |V|/2$) parts.



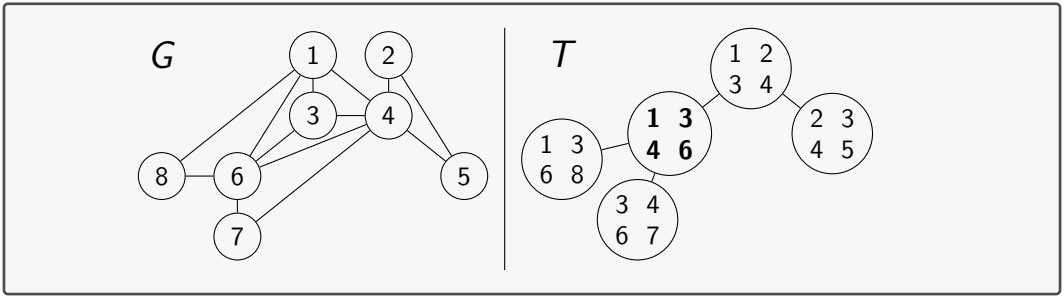
A graph class has bounded treewidth iff it admits small (constant size, balanced) separators.

Small separators

Separators are small sets of vertices that, when removed from the graph, split it into disconnected pieces.

Every bag X_i in T is a separator for G.

You can also always separate G into small ($< |V|/2$) parts.



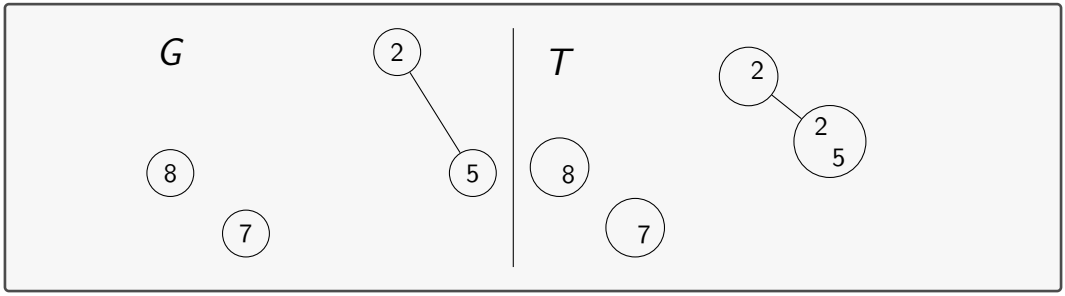
A graph class has bounded treewidth iff it admits small (constant size, balanced) separators.

Small separators

Separators are small sets of vertices that, when removed from the graph, split it into disconnected pieces.

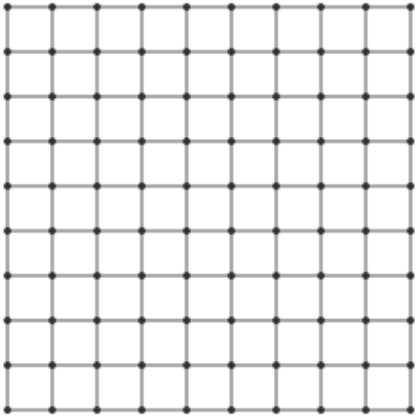
Every bag X_i in T is a separator for G.

You can also always separate G into small ($< |V|/2$) parts.

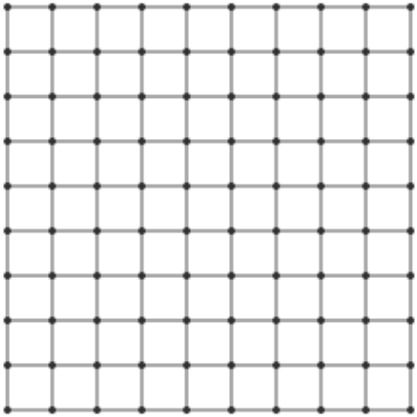


A graph class has bounded treewidth iff it admits small (constant size, balanced) separators.

Does this graph have small treewidth?



Does this graph have small treewidth?



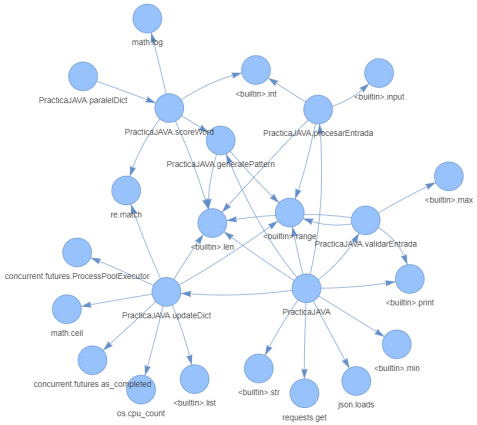
No small separators!

Does this graph have small treewidth?

Does this graph have small treewidth?

Grid-like.

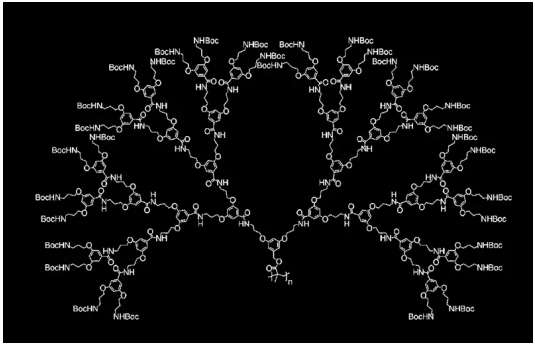
Does this graph have small treewidth?



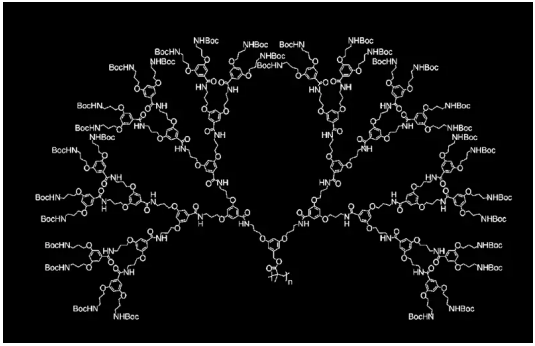
Does this graph have small treewidth?

Yes, but not very small.

Does this graph have small treewidth?

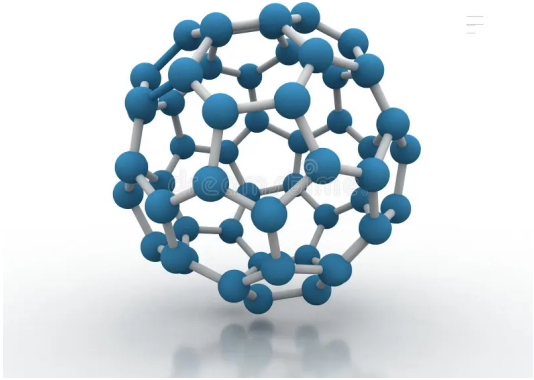


Does this graph have small treewidth?

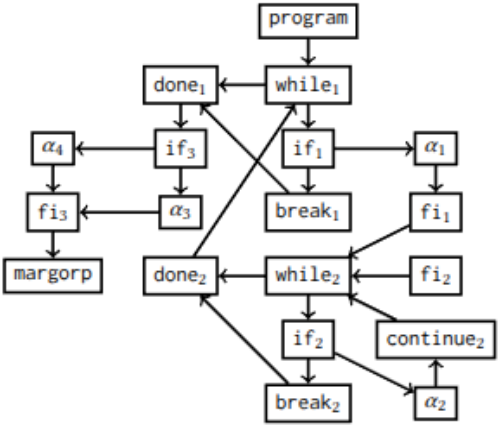


Usually yes, with the exception of grid-like polymers and the football one.

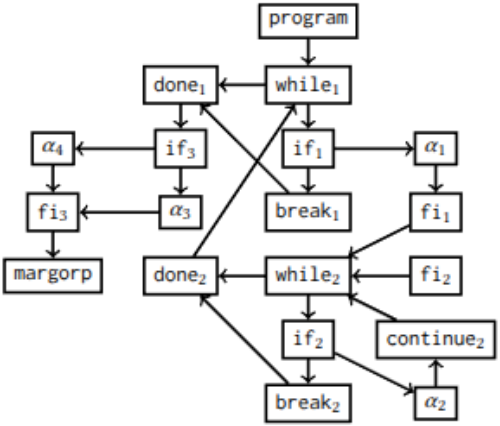
Does this graph have small treewidth?



Does this graph have small treewidth?



Does this graph have small treewidth?



Yes, at most 7 in goto-free programs.

1. What is Treewidth?

2. **Treewidth in Program Analysis**
   - Register Allocation
   - Algebraic Program Analysis
   - Weak Memory Verification

3. Conclusion

Trees

We love trees. Why?

Pros

1. **NP-complete problems can be polynomial on trees** (coloring, smallest vertex cover, largest independent set)
2. **Polynomial problems can be more efficient on trees** (distance queries, dynamic connectivity)
3. **Tree automata allow us to define infinite sets of trees and verify properties of these sets** (reachability in program states)

Cons

1. **They cannot represent anything complicated**

Treewidth is an attempt generalize trees to larger classes of graphs while keeping the pros.

~~Trees~~ Treewidth

We love trees**width**. Why?

Pros

1. **NP-complete problems can be polynomial on trees**width-bounded graphs**
2. **Polynomial problems can be more efficient on trees**width-bounded graphs**
3. **Tree automata allow us to define infinite sets of trees**width-bounded graphs** **and verify properties of these sets**

Cons

1. ~~**They cannot represent anything complicated**~~ There are graphs of unbounded treewidth
2. It does not help with every problem

We will exploit each one of the pros.

Use 1 - Register Allocation

- We must map every variable in a program to a register
- Two variables that are "alive" at the same time (they store a value that can still be used) may not be mapped to the same register
- What is the least amount of registers needed?

Example

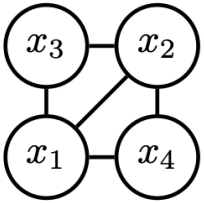
How many registers do we need here?

program	$\{\}$
$x_1 \leftarrow 100\,;$	$\{x_1\}$
$x_2 \leftarrow 40\,;$	$\{x_1, x_2\}$
while $x_2 > 2$ do	$\{x_1, x_2\}$
$x_3 \leftarrow x_1/2$	$\{x_1, x_2, x_3\}$
$x_1 \leftarrow x_1 + x_3$	$\{x_1, x_2, x_3\}$
$x_2 \leftarrow x_2 - x_3$	$\{x_1, x_2, x_3\}$
done ;	$\{x_1, x_2\}$
$x_4 \leftarrow x_1/2 \cdot x_2$	$\{x_1, x_2, x_4\}$
margorp	$\{\}$

From register allocation to a graph problem

- We calculate the live variables at each program point.
- We use this to calculate the **interference graph**. Two variables are adjacent iff they are simultaneously live at some point.

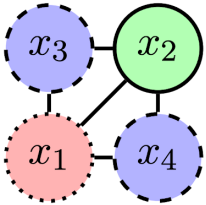
program	$\{\}$
$x_1 \leftarrow 100\,;$	$\{x_1\}$
$x_2 \leftarrow 40\,;$	$\{x_1, x_2\}$
while $x_2 > 2$ do	$\{x_1, x_2\}$
$x_3 \leftarrow x_1/2$	$\{x_1, x_2, x_3\}$
$x_1 \leftarrow x_1 + x_3$	$\{x_1, x_2, x_3\}$
$x_2 \leftarrow x_2 - x_3$	$\{x_1, x_2, x_3\}$
done ;	$\{x_1, x_2\}$
$x_4 \leftarrow x_1/2 \cdot x_2$	$\{x_1, x_2, x_4\}$
margorp	$\{\}$



From register allocation to a graph problem

- Assign variables to registers. Adjacent variables are must be assigned different registers. Minimize registers.
- **This is just graph coloring***.

```
program                 {}
  x1 ← 100;             {x1}
  x2 ← 40;              {x1, x2}
  while x2 > 2 do       {x1, x2}
    x3 ← x1/2           {x1, x2, x3}
    x1 ← x1 + x3        {x1, x2, x3}
    x2 ← x2 − x3        {x1, x2, x3}
  done;                 {x1, x2}
  x4 ← x1/2 · x2        {x1, x2, x4}
margorp                 {}
```



* G. J. Chaitin et al., "Register allocation via coloring," Computer Languages 6(1), pp. 47–57, 1981.

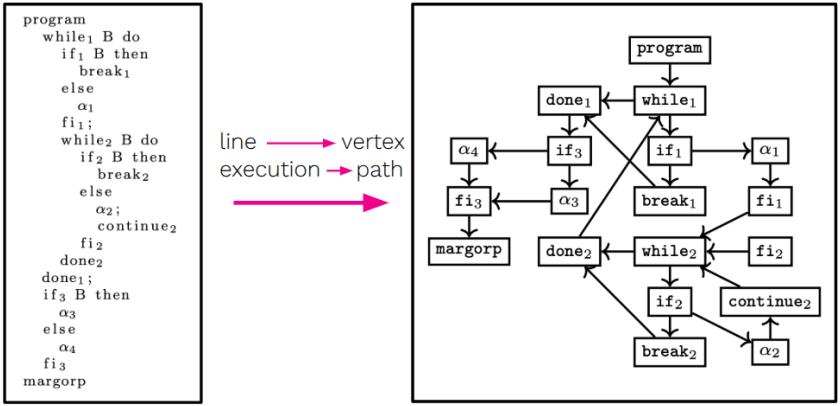
From a graph problem to an easier graph problem

Graph coloring is NP-hard. Let us try to bound the structure of the interference graph.

Let us first look into the program (and its control-flow graph).

From a graph problem to an easier graph problem

A control-flow graph (CFG) is a graph representation of the possible executions of a procedure (flow-sensitive intraprocedural analysis).



From a graph problem to an easier graph problem

The treewidth of CFGs is bounded!!!*

(for go-to free programs)

(the bound is around 7, depends on the language)

We can use this to bound the treewidth of inference graphs.

Graph coloring on bounded-treewidth graphs can be solved in linear time.

* M. Thorup, "All structured programs have small tree width and good register allocation," Information and Computation, 142(2), pp. 159–181, 1998.

How does this impact current research?

Thorup's work opened the door for treewidth-based static analysis.

- G. K. Conrado, A. K. Goharshady, C. K. Lam,
  "The Bounded Pathwidth of Control-Flow Graphs,"
  Proc. OOPSLA 2023.
- K. Chatterjee, A. K. Goharshady, R. Ibsen-Jensen, A. Pavlogiannis,
  "Optimal and Perfectly Parallel Algorithms for On-demand Data-flow Analysis,"
  Proc. ESOP 2020.

What improvement does treewidth bring?

Register Allocation Without Bounded Treewidth

NP-complete

Register Allocation With Bounded Treewidth

Linear time

Use 2 - Boolean Program Analysis

Setup

- Variables are abstracted to Booleans; each CFG edge carries a transformer describing how the variables change when that instruction executes.
- Composing transformers along a path answers: *"if the variables have these values at line s, what are their values at line t?"*

Without Treewidth

- Re-run the full analysis for every query (s, t).
- Cost: $O(n \cdot \alpha(n))$ per query.

With Treewidth

- (CFGs+Call graphs) of real-world programs have small treewidth k.
- Precompute a tree decomposition and cache summaries bottom-up.
- Cost: $O(n \cdot \log n \cdot k^3)$ preprocessing, then $O(k)$ per query.
- About 100 times faster.

Use 2 - Boolean Program Analysis

Boolean program analysis is actually a particular instance of Algebraic Program Analysis, a popular general framework for "consider all paths from s to t" program analysis problems *.

- G.K. Conrado, A.K. Goharshady, K. Kochekov, Y.C. Tsai, A.K. Zaher, "Exploiting the Sparseness of Control-flow and Call Graphs for Efficient and On-demand Algebraic Program Analysis," Proc. OOPSLA 2023.
- K. Chatterjee, R. Ibsen-Jensen, A. Pavlogiannis, and P. Goyal, "Faster Algorithms for Algebraic Path Properties in RSMs with Constant Treewidth", POPL 2015

* Robert Endre Tarjan. 1981. "A Unified Approach to Path Problems". J. ACM 28, 3 (July 1981), 577–593.

What improvement does treewidth bring?

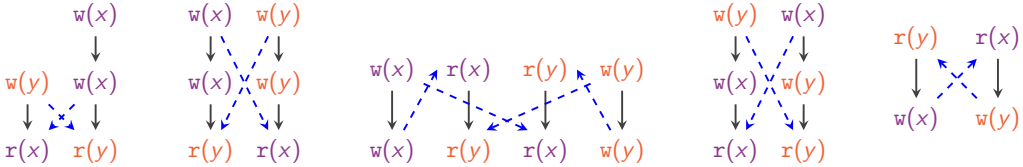
Register Allocation Without Bounded Treewidth	→	Register Allocation With Bounded Treewidth
NP-complete		Linear time

APA Queries Without Bounded Treewidth	→	APA Queries With Bounded Treewidth
Linear		Constant time

Use 3 - Verification Under Weak Memory

Setup

- I can represent program execution through reads-from graphs.
- The weaker the memory model, more reads-from graphs a program can generate.



Use 3 - Verification Under Weak Memory

Setup

- Reads-from graphs of programs under strong memory have bounded treewidth.
- For weak memory (TSO, PSO, WRA, RA, SRA, Relax, etc.), the treewidth is unbounded.

Without Treewidth

- Reachability under RA is undecidable.

With Treewidth

- For executions of bounded treewidth, it is decidable.

Every program either behaves in a very "weak way" (has executions of high treewidth), or has decidable reachability.

Use 3 - Verification Under Weak Memory

This is also part of a general result; properties in Monadic Second-Order Logic (MSO) are decidable for classes of graphs of bounded treewidth.

$$\varphi_{R^+}(e_1, e_2) \triangleq \neg\exists X.\, (e_2 \notin X \wedge \forall e_3, e_4.\, ((e_1 = e_3 \vee e_3 \in X) \wedge R(e_3, e_4) \implies e_4 \in X))$$

We can write RA reachability (given a program, does it have an execution that reaches a certain state) as an MSO formula.

Use 3 - Verification Under Weak Memory

This is a general approach that can be used to treat a variety of verification problems uniformly.

- B. Bollig, C. Di Giusto, A. Finkel, L. Laversa, E. Lozes, and A. Suresh.
  "A Unifying Framework for Deciding Synchronizability"
  CONCUR 2021
- C. Di Giusto, D. Ferré, L. Laversa, and E. Lozes.
  "A Partial Order View of Message-Passing Communication Models"
  POPL 2023
- G. K. Conrado, A. Pavlogiannis.
  "An MSO Framework for Weak-Memory Verification and Robustness" (tentative title)
  Somewhere, soon, hopefully.

Before we went from NP-complete to polynomial and polynomial to better polynomial. Now we went from undecidable to decidable.

What improvement does treewidth bring?

Register Allocation Without Bounded Treewidth NP-complete	→	**Register Allocation With Bounded Treewidth** Linear time
APA Queries Without Bounded Treewidth Linear	→	**APA Queries With Bounded Treewidth** Constant time
RA reachability Without Bounded Treewidth Undecidable	→	**RA reachability With Bounded Treewidth** Decidable

Very versatile!

A recipe for using treewidth

Ask yourself. . .

1. **Is there a graph in your analysis?**
   E.g. a CFG, call graph, interference graph, execution graph.
2. **Is that graph sparse or structured?**
   E.g. comes from structured code, has small separators, has a particular shape.
3. **Is a graph problem the bottleneck?**
   E.g. coloring, reachability, path queries (especially if NP-hard or slow).
4. **Then treewidth might help.**
   NP-hard $\rightarrow$ polynomial $\cdot$ polynomial $\rightarrow$ faster $\cdot$ undecidable $\rightarrow$ decidable.

Come talk to me!

If you answered yes to 1–3, I want to hear about your problem.